

UI Automation Assessment Report

Project Information

Project Name: Selenium Ui Testing

Participant: Tetali Hemanth Venkata Reddy

Date: 29-06-2026

Branch: hemanth_qa_gate2

Assessment: UI Automation using Selenium WebDriver

1. Objective & User Story

Objective

The objective of this automation project is to validate the core user interface functionality of the application using Selenium WebDriver.

The automated test suite validates critical user workflows including navigation, authentication, product search, product selection, cart management, and checkout functionality.

The framework is designed to be scalable, maintainable, and reusable using modern UI automation best practices and the Page Object Model (POM) design pattern.

User Story

As a customer, I want to filter and sort products. So that I can quickly find products matching my preference.

2. Flow Under Test

Scenario – 1:

Search for ""T-Shirts"".

Apply Brand filter.

Verify all displayed products belong to selected brand.

Scenario – 2:

Apply Price filter.

Verify products fall within selected range.

Scenario – 3:

Sort by Price: Low to High.

Verify prices are sorted correctly.

3. Environment & Setup

Application Details

Item - Value

Application URL - Configured through Config.java

Framework - Selenium WebDriver

Language - Java 22

Build Tool - Maven

Test Runner - JUnit 5

Design Pattern - Page Object Model (POM)

Browser - Google Chrome

Driver Management - ChromeDriver

OS - Windows 11

Environment Configuration

Configured using:

```
System.getProperty("BaseURL")
```

```
System.getProperty("headless")
```

Example:

```
BaseURL=
```

```
headless=false
```

Execution Command

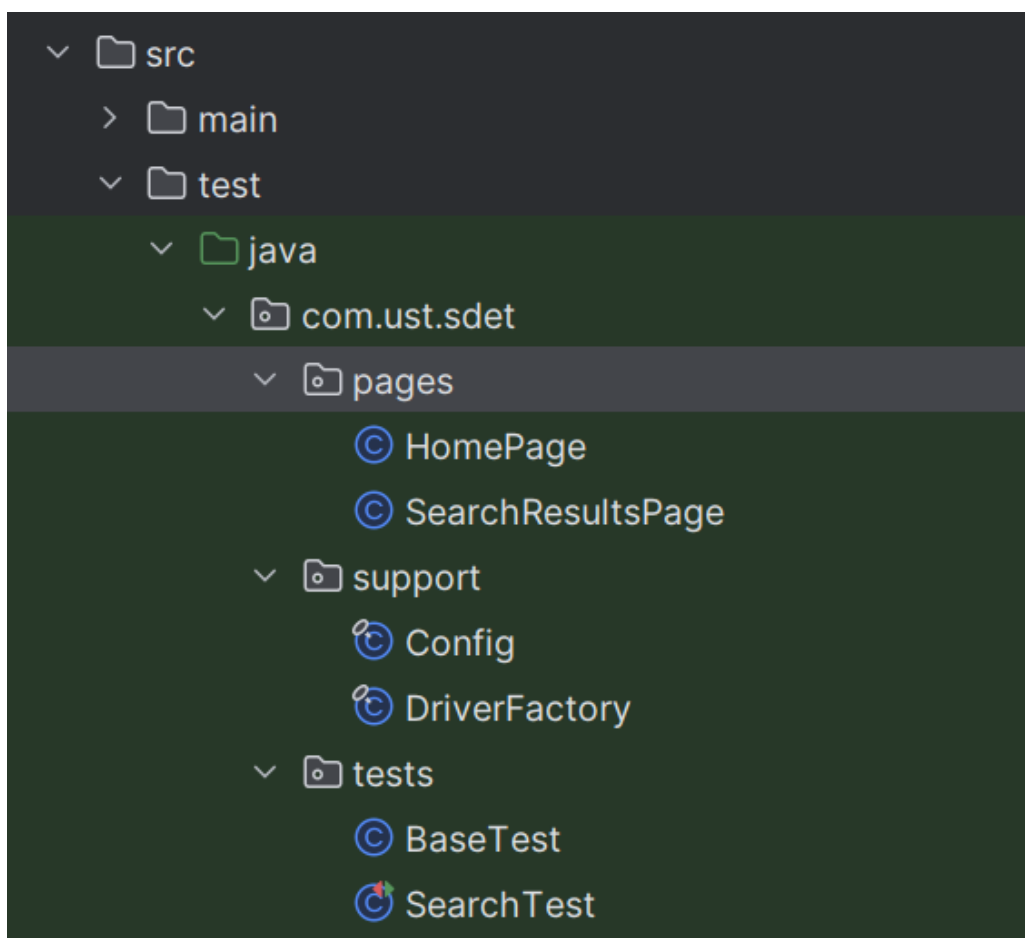
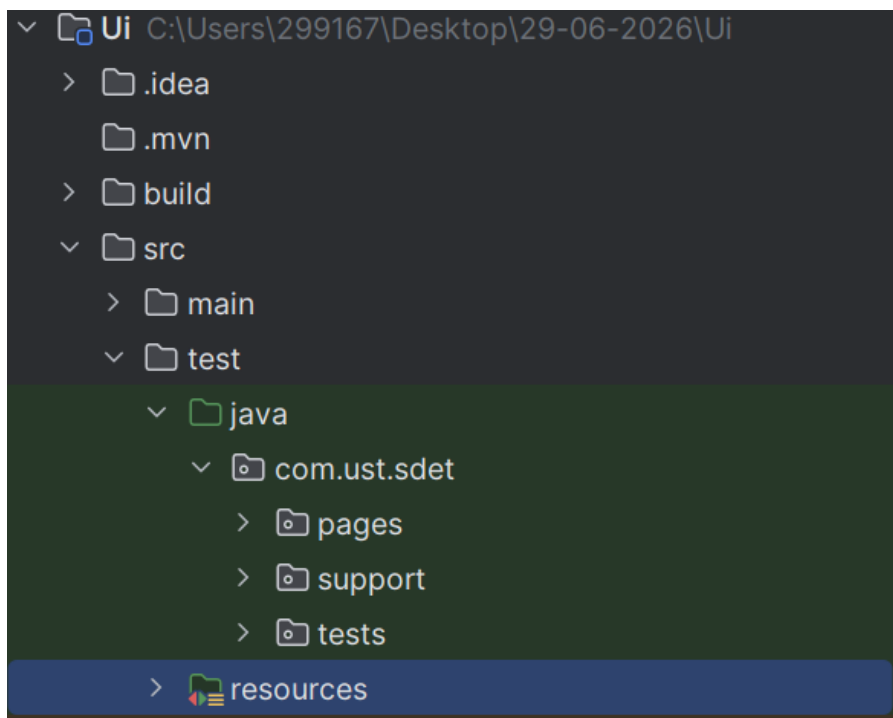
```
mvn clean test
```

Headless Execution:

```
mvn clean test -Dheadless=true
```

4. Framework Design

The framework follows reusable Selenium automation design principles.



5. Test Scenarios Covered

Search for ""T-Shirts"".

Apply Brand filter.

Verify all displayed products belong to selected brand.

Apply Price filter.

Verify products fall within selected range.

Sort by Price: Low to High.

Verify prices are sorted correctly.

6. Automation Best Practices Used

- Selenium WebDriver
- JUnit 5
- Page Object Model (POM)
- Explicit Waits
- Centralized Configuration
- Reusable Driver Factory
- Headless Execution Support
- Clean Test Structure
- Encapsulated UI Actions
- Reusable Utility Methods
- Assertions for UI Elements
- Assertions for Business Flows
- Maven Dependency Management
- Maintainable Framework Design

7. Results



8. Conclusion

The Selenium UI automation framework successfully validates the critical user interface functionality of the application.